ABAP Development Project

Custom ALV Report for Customer-wise Sales Order Analysis

Name	Nayanika Bardhan
Roll Number	2305464
Program	B.Tech Computer Science Engineering
Year	3rd Year
Subject	SAP ABAP Development
Academic Year	2024 – 2025

1. Introduction & Problem Statement

Enterprise Resource Planning (ERP) systems such as SAP generate enormous volumes of transactional data every day. Within the Sales and Distribution (SD) module, sales orders are created, modified and tracked continuously. Business managers and analysts regularly need consolidated, filterable views of this data — grouped by customer, sorted by value, and colour-highlighted for quick decision-making.

Standard SAP transactions like **VA05** (List of Sales Orders) and **VF05** (List of Billing Documents) offer only limited filtering capabilities and provide no automated visual indicators such as colour coding based on order value. Extracting meaningful insights requires toggling between multiple screens, and exporting to Excel is cumbersome in the standard flow.

Problem Statement

Design and implement a **Custom ABAP ALV Report** (*ZNB_SALES_ALV_REPORT*) that fetches sales order data from standard SAP tables, consolidates it with customer master information, provides a powerful selection screen, and displays results in an interactive ALV Grid with subtotals, colour-coded net values, and one-click Excel export.

Scope

- Covers SAP ECC 6.0 and is compatible with S/4HANA.
- Reads data from **VBAK** (Sales Order Header), **VBAP** (Sales Order Items) and **KNA1** (Customer Master – General Data).
- Report is executable via transaction SE38 or a custom transaction code.
- ALV output is exportable to Microsoft Excel via the standard toolbar.

2. Solution & Features

The solution is a single ABAP report program built using the **REUSE_ALV_GRID_DISPLAY_LVC** function module — the recommended ALV framework for SAP ECC — along with supporting data dictionary objects and a custom message class. The design follows ABAP best practices: modular FORM routines, clearly typed internal tables, and no hard-coded values.

2.1 Selection Screen

A dedicated selection screen (Block B1: *Selection Criteria*) provides four SELECT-OPTIONS ranges — Sales Order Number, Customer Number, Order Type, and Creation Date. Block B2 (*Display Options*) offers a configurable row limit and a colour-toggle checkbox. Validation in AT SELECTION-SCREEN ensures the date "from" value never exceeds the "to" value.

2.2 Data Retrieval

A single **SELECT…INTO TABLE** statement performs an INNER JOIN between VBAK and VBAP (linked by sales document number) and a LEFT OUTER JOIN to KNA1 (customer master) to resolve the customer name. The UP TO *p_maxrow* ROWS clause prevents runaway queries on large production systems.

Key fields retrieved:

Field	Source Table	Description
VBELN	VBAK	Sales Order Number
ERDAT	VBAK	Creation Date
AUART	VBAK	Sales Document Type (OR, ZOR…)
KUNNR	VBAK / KNA1	Customer Number
NAME1	KNA1	Customer Name
MATNR / ARKTX	VBAP	Material Number and Description
KWMENG / VRKME	VBAP	Order Quantity and Unit of Measure
NETWR / WAERK	VBAP / VBAK	Net Value and Currency Key

2.3 Colour Coding Logic

After data retrieval (when the colour checkbox is active), the APPLY_COLOR_CODING FORM routine loops through the internal table and populates the *COLOR_FIELD* column

(type LVC_T_SCOL) for each row. The ALV layout parameter *ctab_fname* points to this field so the grid applies per-cell colours automatically:

Condition	Colour	Business Meaning
Net Value ≥ ■10,000	Green (col 5, int 1)	High-value order — priority fulfilment
■1,000 ≤ Net Value < ■10,000	Amber (col 6, int 1)	Medium-value order — standard process
Net Value < ■1,000	Red-inverse (col 6, inv 1)	Low-value order — review or consolidate

2.4 ALV Grid Configuration

- **Zebra striping** (zebra = X) alternates row background for readability.
- **Subtotals** are configured on the KUNNR sort field — each customer block shows an automatic sum of NETWR.
- **Column optimisation** (cwidth_opt = X) auto-sizes every column to its content.
- **Multiple row selection** (sel_mode = A) allows bulk row operations.
- **Layout Variant** /DEFAULT is saved per user for personalisation.

3. Technology Stack

The project is built entirely within the SAP ABAP Workbench using standard SAP-provided tools and frameworks. No third-party add-ons or external libraries are required.

Component	Tool / Object	Purpose
Language	ABAP 7.4+	Core programming language
Development IDE	SAP GUI – SE38 (ABAP Editor)	Write and activate report code
ALV Framework	REUSE_ALV_GRID_DISPLAY_LVC	Interactive grid display
GUI Container	CL_GUI_CUSTOM_CONTAINER	Host ALV grid on screen
DDIC Structure	SE11 – ZNB_SALES_STR	Custom flat structure for internal table
Message Class	SE91 – ZNB_SALES_MSG	Standardised error/info messages
DB Tables Used	VBAK, VBAP, KNA1	Sales orders + customer master
Transport	SE09 / SE10	Move objects across SAP landscapes
Version Control	Git / GitHub	Source code versioning and sharing
Documentation	ReportLab (Python)	Professional PDF generation

SAP Table Relationships

The three database tables involved share the following key relationships:

VBAK (Header)	1	→	N	VBAP (Items)	Linked via VBELN (Sales Doc No.)
VBAK (Header)	N	→	1	KNA1 (Customer)	Linked via KUNNR (Customer No.)

4. Unique Points & Screenshots

4.1 What Makes This Report Distinctive

- **Cell-level colour coding** using LVC_T_SCOL — most student projects use row-level colours (INFO_FNAME). Per-cell colouring on the NETWR field specifically is a more advanced technique requiring deeper knowledge of the LVC framework.
- **Parameterised colour toggle** — the user can switch off colour coding via a checkbox on the selection screen, making the report usable in both colour-capable and monochrome printing contexts.
- **Callback-driven refresh** — the USER_COMMAND callback allows the report to re-execute the database query live without leaving the ALV screen, which is especially useful for monitoring real-time order entry.
- **Subtotals by customer** with sort configuration built programmatically via LVC_T_SORT — this ensures consistent grouping regardless of the order records are retrieved from the database.
- **Macro-based field catalog builder** using the DEFINE…END-OF-DEFINITION construct keeps the catalog code compact and easy to extend without repeating boilerplate.
- **Clean modularisation** — every logical task (fetch, colour, catalog, layout, sort, display) is encapsulated in a named FORM routine with no logic in START-OF-SELECTION beyond orchestration calls.

4.2 Selection Screen (Mockup)

Program: ZNB_SALES_ALV_REPORT – Customer-wise Sales Order Analysis

```
■■ Selection Criteria ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Sales Order         [       ] to [       ]
Customer Number    [       ] to [       ]
Order Type          [       ] to [       ]
Creation Date       [       ] to [       ]
■■ Display Options ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Max Rows            [ 1000 ]
Enable Colours      [✔]
```

[Execute] [Check] [Save]

Figure 1: Selection Screen layout of ZNB_SALES_ALV_REPORT

4.3 ALV Grid Output (Mockup)

Sales Order	Date	Customer	Customer Name	Material	Qty	Net Value	Curr.
0000100001	01.01.2024	1000001	Rajesh Kumar Ent.	MAT-LAPTOP-01	5 EA	**75,000.00**	INR
0000100002	15.02.2024	1000002	Sharma Tech Solutions	MAT-PRINTER-02	2 EA	8,500.00	INR
0000100003	10.03.2024	1000003	Global Imports Ltd.	MAT-CABLE-USB	100 EA	450.00	INR
0000100004	05.04.2024	1000001	Rajesh Kumar Ent.	MAT-MONITOR-24	3 EA	**22,500.00**	INR
			Subtotal – 1000001			**97,500.00**	INR

Figure 2: ALV Grid output with colour-coded Net Value column and customer subtotals

5. Future Improvements

The current implementation provides a solid, production-ready foundation. The following enhancements are planned for subsequent iterations:

1. **Double-click drill-down:** Implement the HANDLE_DOUBLE_CLICK event of CL_GUI_ALV_GRID to open transaction VA03 (Display Sales Order) for the selected row, providing seamless navigation from the report to the source document.

2. **Delivery status enrichment:** JOIN with table VBUK (Sales Document Status) to show overall delivery and billing status against each order, giving a complete order-to-cash picture in a single screen.

3. **Chart integration:** Embed a bar chart using CL_GUI_CHART_ENGINE alongside the ALV grid to visualise customer-wise net value distribution at a glance.

4. **Background scheduling:** Wrap the DB query and output formatting in a background-compatible variant so reports can be scheduled via SM36 and emailed to managers as PDF or Excel attachments.

5. **Purchase Order comparison:** Extend the data model to optionally include purchase orders (EKKO/EKPO) for a combined procurement vs. sales view, useful for supply chain analysis.

6. **Parameterised threshold values:** Move the colour-coding thresholds (currently ■1,000 and ■10,000) to a customising table (Z-table) so business users can adjust them without ABAP changes.

7. **CDS View migration:** Re-implement the data retrieval layer as a Core Data Services (CDS) view for S/4HANA readiness, enabling consumption via OData services and SAP Fiori apps.

6. Conclusion

This project successfully demonstrates the design and implementation of a real-world ABAP ALV report from scratch. Starting from a clearly defined business problem — the absence of a consolidated, colour-coded sales order view — the solution walks through every stage: requirements analysis, data model design, ABAP coding, ALV configuration, and testing.

The report (*ZNB_SALES_ALV_REPORT*) is modular, maintainable, and immediately deployable in any SAP ECC 6.0 system. It demonstrates competency in ABAP Open SQL, function module callbacks, DDIC structures, message classes, and the LVC ALV framework — skills that are directly applicable in SAP SD and ABAP development roles.

Submitted by:

Nayanika Bardhan

Roll No: 2305464 | B.Tech CSE – 3rd Year

Academic Year 2024–2025